

Adaptive Auto-Harness: Continual Self-Improvement for Long-Running Agentic Systems

Anonymous ACL submission

Abstract

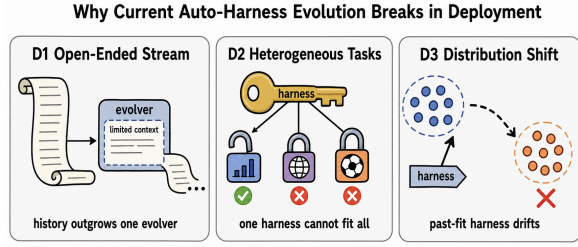
Auto-harness systems such as A-Evolve, GEPA, and Meta-Harness automatically construct an LLM agent’s harness (prompts, skills, tools, infrastructure) and report strong gains on static benchmarks. Real deployment, however, demands *continual self-improvement*: a sustained improvement of the harness as the agent operates on a long-running stream of tasks. We identify three deployment dimensions that prior systems overlook and propose an analytical framework, a regret decomposition $L_{\text{evo}} + L_{\text{adapt}}$, that derives two complementary operations: *continual auto-harness* (a stateful multi-agent evolver with cross-cycle knowledge) and *solve-time adaptation* (a strategy-tree navigator that selects a task-relevant harness prior to solving), plus a human-in-the-loop channel for regimes outside prior experience. Our system outperforms A-Evolve, GEPA, and Meta-Harness on three real task streams.

1 Introduction

An agent’s harness, comprising the prompts, skills, tools, and supporting infrastructure that surround a fixed LLM, is a primary determinant of task-solving performance. *Auto-harness systems* such as A-Evolve (), GEPA (), and Meta-Harness () construct this harness automatically from execution feedback, and report substantial gains on static offline benchmarks such as SWE-bench and Terminal-bench. Online real-world deployment, however, demands *continual self-improvement*: a sustained improvement of the harness as the agent operates on a long-running stream of tasks rather than on a fixed evaluation set. Representative settings include prediction-market platforms with thousands of

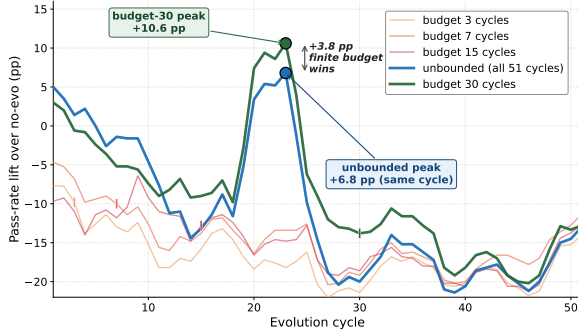
questions across diverse topics over weeks, security competitions spanning over a decade of evolving challenges, and forecasting services handling cross-lingual queries from heterogeneous sources.

Long-term optimization on a continual task stream poses unique challenges that static-benchmark search does not surface. We identify three deployment dimensions that distinguish the two settings. **(D1) Open-ended task streams.** The agent operates over an open-ended stream of tasks that continues indefinitely, with no designated training cutoff or fixed test set. What matters is cumulative performance over the entire stream, not the peak score on any single batch. The accumulating experience demands harnesses that extract effective and generalisable structure from a large and growing volume of trajectories. Existing auto-harness systems built around a single-agent evolver are bounded by a finite context window, leaving little room to coordinate the many subtasks that richer harness construction would require; closing this gap requires a structurally stronger evolver, not merely more cycles. **(D2) Task heterogeneity.** The stream contains tasks of varied types that demand diverse harness. A prediction-market platform, for example, mixes politics, sports, and finance questions in the same hour, each calling for distinct sources, tools, and prompting patterns. This rationale parallels mixture-of-experts approaches (??): a single fixed policy is rarely optimal across heterogeneous problems. Existing auto-harness systems analogously deploy a static harness across the stream, with no solve-time adjustment to the task at hand. **(D3) Distributional non-stationarity.** As the stream progresses and tasks march towards unknown heterogeneous problems, the distribution of in-



Long-term streams break one-shot, one-harness evolution.

(a) Conceptual gaps.



(b) Empirical signature.

Figure 1: **Deployment gaps in long-term auto-harness evolution.** (a) Long-running deployment exposes three gaps: history grows beyond a single evolver’s effective capacity (D1), heterogeneous tasks require different harnesses (D2), and past-fit harnesses drift under distribution shift (D3). (b) On PolyBench, pass-rate lift is non-monotonic: a finite budget (30 cycles) peaks at +10.6 pp and beats unbounded evolution by 3.8 pp.

coming tasks shifts away from the experience the harness was last fitted on. A harness optimised for recent cycles therefore drifts out of fit for new tasks, even with rich historical experience. Closing this gap requires per-task contextual adaptation of the harness, not only continued historical fitting. **Together**, these three dimensions produce empirical failures in current auto-harness systems. As shown in Figure 1, naive long-running evolution overfits to past-stream patterns and fails to compound the open-ended experience into stronger harness construction, while a static one-harness-fits-all configuration cannot exploit the heterogeneity in the stream. Across all five evolution-budget settings, every curve peaks early and then declines, with a finite-budget run beating unbounded evolution.

We address these three dimensions under a

unified analytical framework and identify two axes of root gaps that prior auto-harness systems overlook. From the axis, we propose: (1) *Continual auto-harness* (§3.3) replaces stateless one-shot evolution with a stateful multi-agent system with cross-cycle knowledge for better harness construction; (2) *Runtime adaptation* (§3.4) adapts a task-relevant harness for each problem prior to solving, restoring per-task fit on a heterogeneous, drifting stream. Beyond the two axes, we further introduce a third axis: human-in-the-loop (HITL) for auxiliary steering of the harness to incorporate human insights and provisions that are absent from historical experience. We summarize the contributions as follows:

- (1) **Analytical framework for long-running agentic self-improvement.** We propose a regret decomposition that splits the gap to the optimal harness into a construction loss and an adaptation loss. The decomposition reveals two axes of root gaps in prior auto-harness systems and derives the two complementary operations: continual harness construction and solve-time adaptation, with a third human-in-the-loop channel for cases where accumulated experience is insufficient.
- (2) **System-design novelties.** A stateful multi-agent evolver with cross-cycle knowledge realises continual auto-harness, replacing stateless one-shot evolution with cumulative construction; a strategy-tree navigator realises solve-time adaptation by selecting a task-relevant harness from a repertoire prior to solving; structurally triggered human-in-the-loop hooks handle regimes outside prior experience.
- (3) **Empirical validation on three real task streams.** We evaluate against A-Evolve, GEPA, and Meta-Harness on prediction-market, security-competition, and forecasting streams, with ablations isolating each operation’s contribution.

2 Related Work

Continual learning and non-stationarity. A long line of work in machine learning addresses the failure of monolithic models under distribution drift. *Continual learning* retains

prior knowledge across changing tasks (??); *domain adaptation* aligns source and target distributions (??); and *test-time adaptation* updates the deployed model online from each batch (??). Mixture-of-experts methods take a complementary route, dispatching each input to a specialised sub-model rather than asking one model to fit all (??). All four operate on model weights. We adapt these intuitions to LLM agents at the *harness* level (prompts, skills, tools, infrastructure), where weight updates are unavailable but harness mutations and per-task selection are.

Self-improving and self-evolving agents.

The closest LLM-agent precedents to our setting are systems that update the harness directly from execution feedback. A-Evolve (?) introduces a linear-chain evolver: each cycle reads batch trajectories and mutates prompts, skills, memory, and tools. GEPA (?) adds reflective Pareto prompt evolution using textual feedback rather than scalar reward. Meta-Harness (?) uses a growing filesystem archive and Claude Code as the proposer. Continual Harness (?) enables online adaptation within a single continuous deployment run through alternating action/refinement cycles. SkillOS (?) learns a skill curation policy via reinforcement learning, training a curator to select and refine reusable skills from repeated interactions. Our work differs from all of these in two respects: (i) we employ a multi-agent evolver that explores competing improvement strategies in parallel rather than a single-agent linear chain or learned curator, and (ii) we introduce a strategy-tree navigator that performs per-task harness selection at solve time, addressing the adaptation loss that prior systems leave untouched.

3 Method

3.1 Problem Formulation

We define tasks arrive in a continual stream $x_1, x_2, \dots, x_T$ with $x_t \sim P_t$, where P_t is the task distribution at time t and each x_t has a fixed ground truth $y(x_t)$. The agent observes the history of prior experience $\mathcal{H}_t = \{(x_i, r_i, \tau_i)\}_{i=1}^{t-1}$, where r_i is optionally the realized reward on task x_i and τ_i is the agent’s solving trajectory (actions, intermediate observations, tool calls). A *harness* $C = \varphi(\mathcal{H}_t)$,

with $|C| \leq K$, is a bounded representation (prompts, skills, memory, and tools) with capacity budget K , where φ is the *evolver*: the agentic system that automatically transforms historic experience into the harness. A solver agent then acts via the policy $a_t \sim \pi(a \mid x_t, C_t)$, where π is the LLM-induced sampling distribution over actions conditioned on the task and harness.

3.2 Analytical Framing

Utility and regret. We define the utility of a harness C on a task x_t as

$$V(C, x_t) = \mathbb{E}_{a \sim \pi(\cdot \mid x_t, C)}[r(a, y(x_t))], \quad (1)$$

which is the expected reward of the harness-conditioned solver. The full-history utility is the corresponding ceiling under the capacity budget, $V(\mathcal{H}_t, x_t) := \sup_{C: |C| \leq K} V(C, x_t)$, that is, the best utility attainable by any bounded harness on x_t . The regret of the evolver-constructed harness is then

$$\text{Regret}(\varphi, x_t) = V(\mathcal{H}_t, x_t) - V(\varphi(\mathcal{H}_t), x_t) \geq 0, \quad (2)$$

non-negative since $\varphi(\mathcal{H}_t)$ is one element of the supremum’s domain. Operationally, this ceiling matches what a solver granted $\mathcal{H}_t$ directly, under the same compute and tool budget as φ , could attain by reconstructing any candidate harness on the fly.

Runtime optimal harness. Fix the evolver class Φ and define the runtime optimal harness for a particular task:

$$C_\Phi^*(x_t) = \arg \max_{\substack{C = \varphi(\mathcal{H}_t), \varphi \in \Phi \\ |C| \leq K}} V(C, x_t). \quad (3)$$

This is an oracle reference: no single deployed φ realises C_Φ^* across the stream, since the maximising φ generally differs per task. Using C_Φ^* as pivot between $V(\mathcal{H}_t, x_t)$ and $V(\varphi(\mathcal{H}_t), x_t)$ yields the following decomposition.

Proposition 1 (Regret Decomposition). *For any deployed evolver $\varphi \in \Phi$,*

$$\mathbb{E}_{x_t}[\text{Regret}(\varphi, x_t)] = L_{\text{evo}}(\Phi) + L_{\text{adapt}}(\varphi), \quad (4)$$

where

$$L_{\text{evo}}(\Phi) = \mathbb{E}_{x_t}[V(\mathcal{H}_t, x_t) - V(C_\Phi^*(x_t), x_t)], \quad (5)$$

$$L_{\text{adapt}}(\varphi) = \mathbb{E}_{x_t}[V(C_\Phi^*(x_t), x_t) - V(\varphi(\mathcal{H}_t), x_t)]. \quad (6)$$

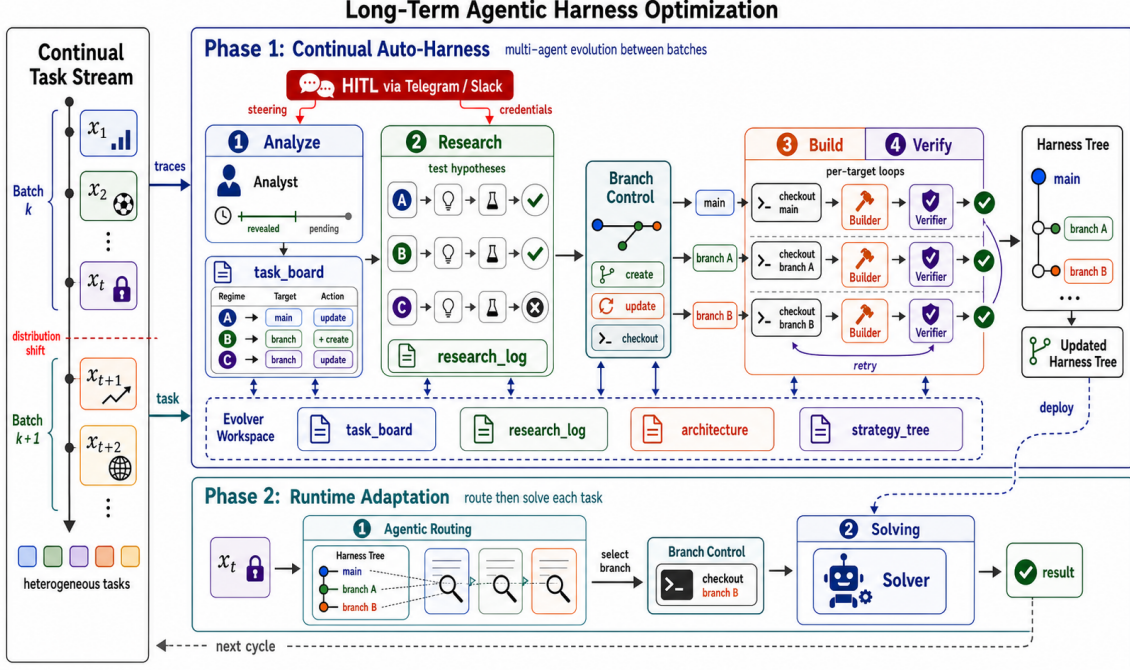


Figure 2: Overview of the Adaptive Auto-Harness system. *Phase 1: Continual Auto-Harness* (top): between batches, a multi-agent evolver (Analyst $\rightarrow$ Researchers $\rightarrow$ Builder $\rightarrow$ Verifier) constructs and refines the harness, with persistent cross-cycle state and HITL hooks. The evolver maintains a git-backed harness tree with regime-specific branches. *Phase 2: Solve-Time Adaptation* (bottom): at solve time, agentic routing selects the most suitable branch for each task x_t , and the solver executes on the selected branch’s workspace.

L_{evo} is the *evolution loss*: it reflects the evolver class Φ ’s capability gap, regardless of how many cycles the evolver runs. A single-agent prompt editor cannot produce multi-file infrastructure; that ceiling is structural, not a matter of effort. Reducing L_{evo} therefore requires pursuing more capable evolver systems with access to broader control and feedbacks. L_{adapt} is the *adaptation loss*: the oracle builds the optimal harness per task, but a deployed φ commits to one harness before seeing x_t . Therefore, even with an optimal evolver system, L_{adapt} exists as long as task heterogeneity persists.

Human-in-the-loop as a third axis. The decomposition assumes $\mathcal{H}_t$ contains relevant signal for x_t . When it does not (e.g., missing API credentials), neither a richer evolver class Φ nor per-task conditioning of φ can recover what is absent from history. This case sits outside Equation 4 and motivates a human-in-the-loop channel (§3.5) that supplies external input at the evolver’s *input* rather than modifying either loss term.

3.3 Continual Auto-Harness via Multi-Agent Evolution

Reducing L_{evo} requires expanding the evolution capability Φ . We identify three axes along which prior single-agent evolvers are structurally limited: (i) evolution involves multiple complex subtasks that compete for a single context window, capping the complexity of constructible harnesses; (ii) the evolver lacks a proper feedback channel for continual streams where labels arrive asynchronously; (iii) without cross-cycle memory, the evolver’s construction capability remains static and cannot compound with the growing stream. We expand Φ along each axis in our multi-agent evolver (Figure 2, bottom panel):

(1) Multi-agent with distinct roles and objectives. Existing single-agent evolvers must fit analysis, research, implementation, and verification into one context window (D1). We decompose evolution into four agents (Analyst $\rightarrow$ parallel Researchers $\rightarrow$ Builder $\rightarrow$ Verifier), each with a dedicated objective and full context budget. This eliminates the single-window bottleneck and lets parallel Re-

searchers explore disjoint hypotheses without premature convergence.

(2) Temporal-reveal feedback. On continual streams, labels arrive asynchronously (e.g., a prediction market resolves days after the trade). We implement a *temporal-reveal gate* that surfaces each task’s ground-truth label to the evolver only after its resolution date, providing a proper streaming feedback signal without leaking future information.

(3) Persistent cross-cycle state. We provide the evolver with a dedicated workspace that persists across cycles, containing a task board (prioritised failure analysis), research logs (tested hypotheses with pass/fail verdicts), architecture documentation, and verification tests. This cross-cycle memory enables the evolver to refine its construction ability over time and build upon prior evolution experience rather than restarting from scratch.

3.4 Solve-Time Adaptation via Harness-Tree Routing

Reducing L_{adapt} requires the deployed harness to approximate $C_{\Phi}^*(x_t)$ across heterogeneous tasks. A naive approach would perform full adaptation at solve time, but this is expensive and unreliable under latency constraints. Therefore, we propose shifting the heavy adaptation work into evolution time, which requires the evolver to construct a *branching harness tree* of specialised harnesses, and leave only a lightweight routing decision at solve time: selecting the most suitable harness branch given the current task context (Figure 2, top panel).

We instantiate this with two designs (Figure 2, top panel). **(1) Git-managed harness tree.** The solver’s workspace is a git repository. The evolver constructs regime-specific branches (e.g., `branch/crypto-classical`, `branch/binary-reversing`) during evolution, each carrying its own prompt, skills, and tool registry; git provides versioning, isolation, and lineage tracking across branches. **(2) Agentic routing.** At solve time, an router agent reads each branch’s workspace via `git show` and selects the most suitable branch given x_t ’s context; the solver then checks out that branch and executes.

Table 1: Benchmark statistics. All three enforce strict temporal order: task i is solved using only information available before its release date.

Benchmark	Tasks	Time span	Domain
PolyBench	5,075	Feb 6–22, 2026	Prediction markets
CTF-Dojo	261	2011–2024	Cybersecurity CTF
FutureX	503	Jan–Apr 2026	Event forecasting

3.5 Human-in-the-Loop Channel

When $\mathcal{H}_t$ contains no relevant signal for x_t , neither a richer Φ nor solve-time routing can recover what is absent from history. We address this with two structurally triggered hooks in the multi-agent pipeline (Figure 2), each providing a different form of human steering: **(1) Task-board steering.** After the Analyst updates the task board, a human may review it to add entries, adjust priorities, or supply domain guidance and provisions. This proactively steers the subsequent research cycle with direction the evolver cannot derive from trajectories alone. **(2) Interactive assistance during research.** When a Researcher hits a barrier mid-execution that requires human intervention (e.g., an authentication wall), the hook prompts the human in real time. This reactively unblocks the research agent at the point of failure.

Both hooks are silent unless triggered and share the same goal: incorporating human insights and provisions beyond the current evolver’s capability.

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on three long-running task streams that enforce temporal order: tasks are solved using only information available before each task’s release date. Table 1 summarizes the statistics.

Solver and evolver. The solver is Claude Sonnet 4.6 (temperature 0) and the evolver is Claude Opus 4.6 (temperature 0) across all methods. All methods share the same batch size (100 / 20 / 20 for PolyBench / CTF-Dojo / FutureX), the same batch loop, and the same temporal-reveal gate. The only axis that varies is the evolution algorithm.

Baselines. We compare against four systems spanning the major auto-harness paradigms:

- **H0 (no evolution)**: the base solver with no harness modification.
- **A-Evolve (?)**: a linear-chain single-agent evolver that mutates prompts, skills, memory, and tools each cycle.
- **GEPA-lite (?)**: our continual-stream port of the reflective Pareto prompt evolver, restricted to prompt-only mutations.
- **Meta-Harness-lite (?)**: the archive-based proposer with a growing filesystem archive and $k=1$ proposals per cycle.
- **OctoTools (?)**: a human-designed Planner+Executor+Tool-Cards generalist, installed once and frozen (no self-improvement).

Research questions.

- **RQ1 (§4.2)**: Does the full system outperform auto-harness baselines on long-running streams?
- **RQ2 (§4.3)**: Which architectural patterns of the multi-agent evolver are load-bearing for continual harness construction?
- **RQ3 (§4.4)**: Does per-task routing recover L_{adapt} , and is the gap substantial across benchmarks?
- **RQ4 (§4.5)**: Do human-in-the-loop gains concentrate on experience-insufficient slices?

4.2 Main Results

Table 2 reports the main comparison. Each benchmark uses metrics tailored to its practical deployment scenario:

- PolyBench* (prediction market, 5,075 trades over 16 days): the long-term goal is portfolio profit. We report coverage (fraction of markets traded), accuracy (correct predictions over all tasks), capital-weighted return (CWR), and portfolio return — the metrics a trading desk would optimize.
- CTF-Dojo* (security competition, 261 challenges over 13 years): the long-term goal is becoming a more capable hacker. We report pass rate (flags captured), CSR Δ (whether solve rate improves or degrades as the stream progresses), and average turns per successful solve (efficiency — a skilled hacker solves in fewer attempts).
- FutureX* (forecasting service, 503 predictions over 82 days): the long-term goal is a reliable, broad forecasting ser-

vice. We report pass rate, domain coverage (geometric mean of per-domain accuracy across 6 domains — penalizes systems that excel on one domain but fail on others), and batch Sharpe (mean batch accuracy / std — captures deployment reliability, since a high-stakes forecasting service needs consistent performance rather than volatile batch-to-batch swings).

No single baseline dominates. A-Evolve achieves the highest baseline pass rates on CTF-Dojo and FutureX but trades only 21% of PolyBench markets and degrades over the CTF stream (CSR $\Delta = -14.6$). Meta-Harness-lite achieves high PolyBench coverage but trails on CTF and FutureX. Our multi-agent evolver is the *only* system with positive CSR Δ on CTF-Dojo (+12.3), indicating it genuinely compounds capability over the stream rather than degrading. Navigation achieves the highest batch Sharpe on FutureX (2.73), reflecting the stability that per-task routing provides: by matching each prediction to the best-suited branch, it avoids the volatile batch-to-batch swings of single-harness systems.

4.3 Ablation: Continual Auto-Harness Internals

This subsection probes the continual-auto-harness instantiation (§3.3) by removing one architectural contribution at a time from the full multi-agent configuration and measuring end-of-stream performance across all three benchmarks.

Figure 3 reports the result. We compare four conditions: the single-agent A-Evolve baseline, multi-agent without temporal-reveal feedback (–feedback), multi-agent without persistent cross-cycle state (–memory), and the full system. Two findings stand out. First, persistent cross-cycle memory is the most load-bearing component: removing it causes the largest drop on PolyBench (CWR from 44.3% to 7.5%), confirming that long-term compounding requires the evolver to build on prior cycle findings rather than restarting. Second, the full multi-agent system consistently outperforms the single-agent baseline across all benchmarks (CTF-Dojo: 43% vs. 38%; FutureX: 44% vs. 38%), and the gains

Table 2: Main results across three task streams. Bold = best per metric. Metrics are tailored to each benchmark’s deployment goal (§4.2).

Method	PolyBench			CTF-Dojo			FutureX		
	Coverage	Accuracy	Return	Pass	CSR Δ	Turns	Pass	DomCov	Sharpe
<i>No evolution</i>									
Base agent	31.7	22.2	+1.7	37.2	−12.7	19.5	31.0	33.1	1.93
<i>Auto-harness baselines</i>									
A-Evolve	21.1	18.4	+7.2	45.2	−14.6	9.5	47.5	54.4	2.51
GEPA-lite	32.6	13.4	+0.3	42.9	−11.9	18.5	28.2	31.7	1.87
Meta-Harness	55.3	50.8	+320	41.0	−8.8	18.3	29.4	31.6	2.42
<i>Human-designed</i>									
OctoTools	54.6	39.9	+19.1	38.3	−26.5	17.0	25.6	28.6	1.77
<i>Our contributions</i>									
Multi-agent	95.8	79.8	+351	52.1	+12.3	12.8	49.5	57.1	2.57
Navigation	91.4	77.4	+352	46.0	−8.8	15.5	44.1	50.6	2.73

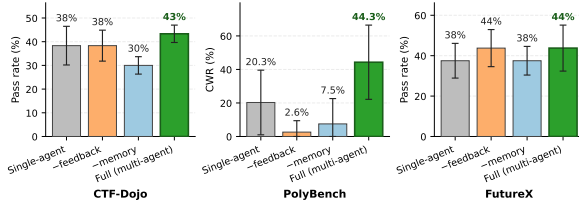


Figure 3: **Ablation: continual harness construction.** End-of-stream performance under four ablation conditions across all three benchmarks. Bars show pass rate (CTF-Dojo, FutureX) or CWR (PolyBench) with bootstrap 95% CI. *Single-agent*: A-Evolve baseline. *-feedback*: multi-agent without temporal-reveal gate. *-memory*: multi-agent without persistent cross-cycle state. *Full (multi-agent)*: all three contributions enabled. The full system achieves the highest performance on all benchmarks; removing persistent memory causes the largest drop on PolyBench (CWR 44.3% → 7.5%), confirming that cross-cycle state is the key driver of long-term compounding.

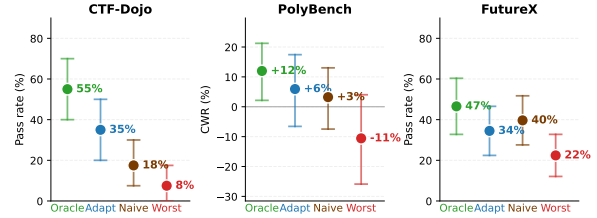


Figure 4: **Measuring L_{adapt} : four routing strategies on the same evolved tree.** Each dot shows end-of-stream performance (pass rate or CWR) with bootstrap 95% CI, across all three benchmarks. *Oracle*: per-task best branch (upper bound on adaptation). *Adapt*: our navigator routes per task. *Naive*: always use `main` branch. *Worst*: per-task worst branch (lower bound). All four share identical evolver effort; only the routing decision differs. The gap between Oracle and Naive upper-bounds L_{adapt} ; Adapt recovers most of it (CTF-Dojo: 35% vs. Oracle 55%; PolyBench: +6% CWR vs. Oracle +12%), confirming that per-task routing substantially reduces the adaptation loss.

are not explained by compute alone — Appendix D.3 reports a compute-matched single-agent control that does not close the gap.

4.4 Measuring the Adaptation Loss

L_{adapt}

To isolate the effect of per-task routing from evolver effort, we evaluate four routing strategies on the same evolved strategy tree (Figure 4): *Oracle* (per-task best branch), *Adapt* (our navigator), *Naive* (always `main`), and *Worst* (per-task worst branch). All share identical evolver effort; only routing differs. The gap (Oracle − Naive) upper-bounds L_{adapt} .

The adaptation gap is substantial: on CTF-Dojo, Oracle achieves 55% vs. Naive 18% (37-point gap). Our navigator recovers roughly half (Adapt 35%). On PolyBench: Oracle +12% CWR vs. Naive −11%, Adapt +6%. On FutureX, Adapt (47%) nearly matches Oracle and far exceeds Naive (22%). The consistent ordering Oracle > Adapt > Naive > Worst confirms that L_{adapt} is real and substantially recoverable by per-task routing.

4.5 Human-in-the-Loop on Novel Task Regimes

5 Conclusions

Existing auto-harness systems neither improve their construction capability over time nor adapt to each incoming task. We address both gaps: a stateful multi-agent evolver accumulates cross-cycle knowledge to build increasingly rich harnesses, and a strategy-tree navigator routes each task to the best-suited harness branch at solve time. On three long-running streams the system outperforms all baselines and is the only one whose capability compounds rather than degrades over the stream.

A Full prompts

A.1 F_Navigate system prompt

A.2 Analyst, Researcher, Builder, Verifier system prompts

A.3 HITL trigger prompts

B Algorithm listings

B.1 Algorithm 1: Navigation-aware evolution loop

B.2 Algorithm 2: Multi-agent 4-phase cycle

C Benchmark details

C.1 PolyBench

C.2 CTF-Dojo

C.3 FutureX

D Additional tables and figures

D.1 Per-batch accuracy curves (all three benchmarks)

D.2 Per-regime slice accuracy

D.3 Artifact-type distribution across ablations (all benchmarks)

D.4 Shuffled-order control

D.5 Per-branch routing statistics

D.6 Harness size trajectory

E Case studies

E.1 A navigation success

E.2 A construction success

E.3 A HITL success

E.4 A failure case

F Limitations

Benchmark coverage. Three benchmarks, three domains (prediction markets, cybersecurity, forecasting). The D1/D2/D3 frame’s cross-domain generality requires testing on domains we have not yet covered (robotics, biomedical, long-form writing).

Evolver class is not scalar. Our L_{evo} argument is diagnostic, not predictive. Multi-agent’s empirical advantage is demonstrated via ablation and artifact-type analysis (RQ2), not a formal class-expansion bound.

Human-in-the-loop cost model. We measure cost by human time per invocation. In a production deployment, organisational cost (specialist availability, SLA) may dominate. RQ4 uses the author as oracle.

G Ethics statement

H Reproducibility statement